

Lecture 8

Tuesday February 17th 2026

- Quiz 1 is on Thursday will need to schedule time with the TA
- Project 1 is starting and need to form a group of 3
 - Create a Unix shell
 - runs as a while loop that waits for input from the user
 - process the input
 - search for program
 - `fork()`
 - flash it with `execvp(ls)` - (ls is the search result)
 - will make the child execute the `ls` program
- `wish.c`
- `gcc -o wish wish.c`
- `./wish`
- run from the terminal and run your program from there
- need to implement a few basic commands `ls`, `cd`, `exit`, `path`.
- need to be able to run parallel commands using `&`
 - create separate forks for each command
- error handling will be a single error message that will pop up for any error
- have until midterm to complete 3 weeks

Threads

- Necessary for parallel processing
- requires multiple CPUs or a GPU in order to be true parallelism
- a thread is the execution path of a program
- tabs in a browser are different threads
- A process is **heavyweight**
- a thread is **lightweight**
- Threads are much quicker to create than a process
- faster to switch between threads than processes
- threads share data more easily

Exercise

- slide 25

- execution of two threads and the value of the two variables at the end of execution
- OS can decide either thread goes first and the answer changes based on which thread access the variables and writes over the other
- Parallel programs can be very uncertain so you must program for deterministic actions

Threading Examples

- **Web server**
 - Each new request comes in a "dispatcher thread" spawns a new "worker thread" to read the requested file
 - Worker threads may be discarded or recycled in a "thread pool"

Threading Implementation

- Two broad categories:
 - User-Level Threads **ULT**
 - Kernel-Level Threads **KLT**
 - Can also do a hybrid or combined model using both

One-to-One Model

- Each user-level thread maps to a kernel

Threading Issues

- Thread pools
- Thread specific data
- Semantics of `fork()` and `exec()` system calls
- Thread cancellation